LAB ASSIGNMENT-17.5

NAME : RISHIK.P

HALL TICKET NO : 2303A51778

BATCH : 12

Task 1 –Loan Approval Dataset Cleaning

Use AI to generate preprocessing code for a loan dataset.

Instructions:
• Handle missing values in loan_amount, credit_score.
• Encode loan_status (Approved/Rejected).
• Scale numerical features using standard scaling.
• Split the dataset into training and testing sets.

Sample Code:

```
import pandas as pd

data = pd.read_csv("loadapproval.csv")

print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/amineipad/loan-approval-dataset

Expected Output:
• A machine-learning-ready loan dataset.

PROMPT

"""Generate Python code to preprocess a loan approval dataset.

Load a CSV file named 'loanapproval.csv'.

Handle missing values in 'loan_amount' and 'credit_score' using mean imputation.

Encode the 'loan_status' column (Approved/Rejected) using label encoding.

Scale numerical features using StandardScaler.

Split the dataset into training and testing sets (80/20).

Save the cleaned dataset as 'loanapproval_cleaned.csv'"""

CODE

```python
Save the cleaned dataset as 'loanapproval_cleaned.csv'"""

import pandas as pd
import os
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Get the directory of the current script
base_dir = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(base_dir, "loanapproval.csv")

# Load the dataset
df = pd.read_csv(file_path)

print("Original Dataset:")
print(df.head())
print("\nColumns:", df.columns)

# Handle missing values
df['loan_amount'] = df['loan_amount'].fillna(df['loan_amount'].mean())
df['credit_score'] = df['credit_score'].fillna(df['credit_score'].mean())

# Encode categorical variables
df = pd.get_dummies(
    df,
    columns=['gender', 'marital_status', 'employment_status'],
    drop_first=True
)

# Scale numerical features
scaler = StandardScaler()
numerical_features = [
    'age', 'annual_income', 'loan_amount',
    'credit_score', 'num_dependents', 'existing_loans_count'
]
df[numerical_features] = scaler.fit_transform(df[numerical_features])

# Define features and target
X = df.drop(['loan_approved', 'applicant_id'], axis=1)
y = df['loan_approved']

# Split the dataset into training and testing sets (80/20)
```

OUTPUT

```
PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/Rishik/OneDrive/Desktop/AI/assignment17.5/
task1_loan.py
Original Dataset:
   applicant_id  age  gender marital_status  annual_income  loan_amount  credit_score  num_dependents  existing_loans_count employment_status  loan_approved
0             1   59    Male       Divorced         100073         7169           793               1                     1        Unemployed              1
1             2   49    Male        Married         112197        23556           789               0                     2          Employed              1
2             3   35    Male       Divorced          84429        27052           372               1                     4        Unemployed              0
3             4   63  Female         Single         124195        11313           808               3                     4     Self-employed              1
4             5   28  Female        Married          81627        13315           689               0                     1        Unemployed              1

Columns: Index(['applicant_id', 'age', 'gender', 'marital_status', 'annual_income',
       'loan_amount', 'credit_score', 'num_dependents', 'existing_loans_count',
       'employment_status', 'loan_approved'],
      dtype='str')

Training Set Shape: (800, 11)
Testing Set Shape: (200, 11)
```

EXPLANATION

The loan approval dataset was preprocessed to make it suitable for machine learning. Missing values in the loan_amount and credit_score columns were handled using mean imputation to maintain data consistency. Categorical variables such as gender, marital_status, and employment_status were encoded using one-hot encoding, while the target variable loan_approved was used for prediction. Numerical features were standardized using StandardScaler, and the dataset was split into training and testing sets in an 80:20 ratio. The cleaned dataset was saved as loanapproval_cleaned.csv.

Task 2 – Social Media Profiles

Preprocess a social media engagement dataset using AI assistance.

Instructions:
• Remove duplicate user profiles.
• Handle missing values
• Encode categorical columns

Dataset link:

https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles

Expected Output:
• A cleaned dataset for engagement prediction.

PROMPT

"""Generate Python code to preprocess a social media engagement dataset.

Load a CSV file named 'social_media_profiles.csv'.

Remove duplicate user profiles.

Handle missing values appropriately (use mean for numerical columns and mode for categorical columns).

Encode categorical columns using Label Encoding.

Save the cleaned dataset as 'social_media_cleaned.csv'.

Display the dataset shape before and after cleaning."""

CODE

```python
assignment17.5 > task2_social_media.py > ...
import pandas as pd
import os
from sklearn.preprocessing import LabelEncoder

# Get the directory of the current script
base_dir = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(base_dir, "social_media_profiles.csv")

# Load dataset with proper encoding
df = pd.read_csv(file_path, encoding='latin1')

# Clean column names (remove extra spaces and hidden characters)
df.columns = df.columns.str.strip().str.replace('\xa0', '', regex=True)

print("Original Dataset Shape:", df.shape)
print(df.head())
print("\nColumns:", df.columns)

# Remove duplicate profiles
df.drop_duplicates(inplace=True)

# Convert numeric columns to proper numeric dtype
numeric_cols = ['FOLLOWERS', 'POSTS']
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Handle missing values
# Numerical columns → mean
for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())

# Categorical columns → mode
categorical_cols = ['PROFILE', 'Platform']
for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

# Encode categorical variables using LabelEncoder
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le
```

OUTPUT

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/Rishik/On
task2_social_media.py
Original Dataset Shape: (500, 5)
   N             PROFILE    FOLLOWERS   POSTS    Platform
0  1           Instagram   539446645.0  7202.0  Instagram
1  2   Cristiano Ronaldo   473864939.0  3338.0  Instagram
2  3  Kylie <U+0001F90D>   364542529.0  6935.0  Instagram
3  4           Leo Messi   355790796.0   890.0  Instagram
4  5        Selena Gomez   341579063.0  1828.0  Instagram

Columns: Index(['N', 'PROFILE', 'FOLLOWERS', 'POSTS', 'Platform'], dtype='str')

Cleaned Dataset Shape: (500, 4)
✅ Cleaned dataset saved as 'social_media_cleaned.csv'
   PROFILE    FOLLOWERS   POSTS  Platform
0      151  539446645.0  7202.0         0
1       86  473864939.0  3338.0         0
2      204  364542529.0  6935.0         0
3      216  355790796.0   890.0         0
4      323  341579063.0  1828.0         0
PS C:\Users\Rishik\OneDrive\Desktop\AI>
```

EXPLANATION

The social media profiles dataset was cleaned to prepare it for engagement analysis. Duplicate user entries were removed to ensure data uniqueness. Numerical columns like FOLLOWERS and POSTS were converted to numeric types and missing values were imputed using their mean, while categorical columns such as PROFILE and Platform were filled with the mode. These categorical features were then encoded

using LabelEncoder to make them suitable for machine learning models. The processed dataset was saved as social_media_cleaned.csv.

Task 3 – Weather Dataset Feature Engineering

Task:
Use AI to preprocess a weather dataset for forecasting.

Instructions:
• Convert date column into datetime format.
• Handle missing values in temperature, humidity.
• Create new features (season, week_number).
• Normalize rainfall values.

Dataset link:

https://www.kaggle.com/code/alsamaualalhinai/weather-dataset/input

Expected Output:
• A processed dataset ready for forecasting models.

PROMPT

"""Generate Python code to preprocess a weather dataset.

Load 'weather-dataset.csv'.

Convert the 'Date_Time' column to datetime format.

Handle missing values in temperature and humidity using mean imputation.

Create new features: 'season' and 'week_number'.

Normalize precipitation (rainfall) values using MinMaxScaler.

Save the processed dataset as 'weather_processed.csv'."""

CODE

```python
import pandas as pd
import os
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
base_dir = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(base_dir, "weather-dataset.csv")
# Load the dataset
df = pd.read_csv(file_path)
print("Original Dataset:")
print(df.head())
print("\nColumns:", df.columns)
# Convert 'date' column to datetime format
df['date'] = pd.to_datetime(df['date'])
# Handle missing values
numeric_cols = ['precipitation', 'temp_max', 'temp_min', 'wind']
for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())
# Create new feature: week_number
df['week_number'] = df['date'].dt.isocalendar().week
# Create new feature: season
def get_season(month):
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Summer'
    elif month in [6, 7, 8]:
        return 'Monsoon'
    else:
        return 'Autumn'
df['season'] = df['date'].dt.month.apply(get_season)
# Normalize rainfall (precipitation) values between 0 and 1
scaler = MinMaxScaler()
df['precipitation'] = scaler.fit_transform(df[['precipitation']])
# Encode the categorical 'weather' column
le = LabelEncoder()
df['weather'] = le.fit_transform(df['weather'])
# Save the processed dataset
output_path = os.path.join(base_dir, "weather_processed.csv")
df.to_csv(output_path, index=False)

print("\n✅ Weather dataset processed and saved as 'weather_processed.csv'")
print("\nProcessed Dataset:")
print(df.head())
```

OUTPUT

```
PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/Rishik/Or
task3_weather.py
Original Dataset:
         date  precipitation  temp_max  temp_min  wind  weather
0  2012-01-01            0.0      12.8       5.0   4.7  drizzle
1  2012-01-02           10.9      10.6       2.8   4.5     rain
2  2012-01-03            0.8      11.7       7.2   2.3     rain
3  2012-01-04           20.3      12.2       5.6   4.7     rain
4  2012-01-05            1.3       8.9       2.8   6.1     rain

Columns: Index(['date', 'precipitation', 'temp_max', 'temp_min', 'wind', 'weather'], dtype='str')

✅ Weather dataset processed and saved as 'weather_processed.csv'

Processed Dataset:
        date  precipitation  temp_max  temp_min  wind  weather  week_number  season
0 2012-01-01       0.000000      12.8       5.0   4.7        0           52  Winter
1 2012-01-02       0.194991      10.6       2.8   4.5        2            1  Winter
2 2012-01-03       0.014311      11.7       7.2   2.3        2            1  Winter
3 2012-01-04       0.363148      12.2       5.6   4.7        2            1  Winter
4 2012-01-05       0.023256       8.9       2.8   6.1        2            1  Winter
PS C:\Users\Rishik\OneDrive\Desktop\AI>
```

EXPLANATION

The weather dataset was preprocessed to enhance its suitability for forecasting tasks. The date column was converted into a datetime format, enabling time-based feature extraction. Missing values in numerical columns such as precipitation, temp_max, temp_min, and wind were handled using mean imputation. New features, including week_number and season, were created to capture temporal patterns in the data. Additionally, precipitation values were normalized using MinMaxScaler, and

the categorical weather column was encoded. The final dataset was saved as weather_processed.csv.

Task 4 – Movie Ratings Dataset Preparation

Task:
Clean and preprocess a movie ratings dataset using AI-generated scripts.

Instructions:
• Handle missing values in rating, genre.
• Encode categorical variables like genre, language.
• Remove duplicate movie entries.
• Scale rating values between 0 and 1.

Dataset link:

https://www.kaggle.com/datasets/PromptCloudHQ/imdb-data

Expected Output:
• A dataset suitable for recommendation systems.

PROMPT

"""Generate Python code to preprocess a movie ratings dataset.

Load a CSV file named 'imdb_data.csv'.

Remove duplicate movie entries based on the 'Title' column.

Handle missing values in 'Rating', 'Genre', and 'Language'.

Encode categorical variables such as 'Genre' and 'Language'.

Normalize the 'Rating' column between 0 and 1 using MinMaxScaler.

Save the cleaned dataset as 'movie_ratings_cleaned.csv'."""

CODE

```
ssignment17.5 > task4_movie.py > ...
Remove duplicate movie entries based on the 'Title' column.
Handle missing values in 'Rating', 'Genre', and 'Language'.
Encode categorical variables such as 'Genre' and 'Language'.
Normalize the 'Rating' column between 0 and 1 using MinMaxScaler.
Save the cleaned dataset as 'movie_ratings_cleaned.csv'."""

import pandas as pd
import os
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
base_dir = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(base_dir, "imdb_data.csv")
# Load the dataset
df = pd.read_csv(file_path, encoding='latin1')
print("Original Dataset Shape:", df.shape)
print(df.head())
print("\nColumns:", df.columns)
# Remove duplicate movie entries based on 'Title'
df.drop_duplicates(subset='Title', inplace=True)
# Handle missing values
df['Rating'] = df['Rating'].fillna(df['Rating'].mean())
df['Genre'] = df['Genre'].fillna('Unknown')
df['Revenue (Millions)'] = df['Revenue (Millions)'].fillna(
    df['Revenue (Millions)'].mean()
)
df['Metascore'] = df['Metascore'].fillna(df['Metascore'].mean())
# Encode the 'Genre' column
le_genre = LabelEncoder()
df['Genre'] = le_genre.fit_transform(df['Genre'])
# Normalize the 'Rating' column between 0 and 1
scaler = MinMaxScaler()
df['Rating'] = scaler.fit_transform(df[['Rating']])
# (Optional) Drop columns not required for modeling
# df.drop(['Description', 'Director', 'Actors'], axis=1, inplace=True)
print("\nCleaned Dataset Shape:", df.shape)
# Save the cleaned dataset
output_path = os.path.join(base_dir, "movie_ratings_cleaned.csv")
df.to_csv(output_path, index=False)
print("✅ Movie ratings dataset saved as 'movie_ratings_cleaned.csv'")
print(df.head())
```

OUTPUT

```
PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/Rishik/OneDrive/Desktop/AI/assignment17.5/
task4_movie.py
Original Dataset Shape: (1000, 12)
   Rank                  Title                   Genre                                       Description  ... Rating   Votes  Revenue (Millions)  Metascore
0     1  Guardians of the Galaxy   Action,Adventure,Sci-Fi  A group of intergalactic criminals are forced ...  ...    8.1  757074              333.13       76.0
1     2              Prometheus  Adventure,Mystery,Sci-Fi  Following clues to the origin of mankind, a te...  ...    7.0  485820              126.46       65.0
2     3                   Split           Horror,Thriller  Three girls are kidnapped by a man with a diag...  ...    7.3  157606              138.12       62.0
3     4                    Sing   Animation,Comedy,Family  In a city of humanoid animals, a hustling thea...  ...    7.2   60545              270.32       59.0
4     5            Suicide Squad  Action,Adventure,Fantasy  A secret government agency recruits some of th...  ...    6.2  393727              325.02       40.0

[5 rows x 12 columns]

Columns: Index(['Rank', 'Title', 'Genre', 'Description', 'Director', 'Actors', 'Year',
       'Runtime (Minutes)', 'Rating', 'Votes', 'Revenue (Millions)',
       'Metascore'],
      dtype='str')

Cleaned Dataset Shape: (999, 12)
✅ Movie ratings dataset saved as 'movie_ratings_cleaned.csv'
   Rank                  Title  Genre                                       Description  ...    Rating   Votes  Revenue (Millions)  Metascore
0     1  Guardians of the Galaxy     11  A group of intergalactic criminals are forced ...  ...  0.873239  757074              333.13       76.0
1     2              Prometheus     85  Following clues to the origin of mankind, a te...  ...  0.718310  485820              126.46       65.0
2     3                   Split    195  Three girls are kidnapped by a man with a diag...  ...  0.760563  157606              138.12       62.0
3     4                    Sing     92  In a city of humanoid animals, a hustling thea...  ...  0.746479   60545              270.32       59.0
4     5            Suicide Squad      7  A secret government agency recruits some of th...  ...  0.605634  393727              325.02       40.0

[5 rows x 12 columns]
PS C:\Users\Rishik\OneDrive\Desktop\AI>
```

EXPLANATION

The movie ratings dataset was cleaned and prepared for use in recommendation systems. Duplicate movie entries were removed based on the Title column to ensure data integrity. Missing values in Rating, Genre, Revenue (Millions), and Metascore were handled using appropriate imputation techniques. The categorical Genre column was encoded using LabelEncoder, and the Rating feature was normalized to a range between 0 and 1 using MinMaxScaler. The resulting machine-learning-ready dataset was saved as movie_ratings_cleaned.csv.